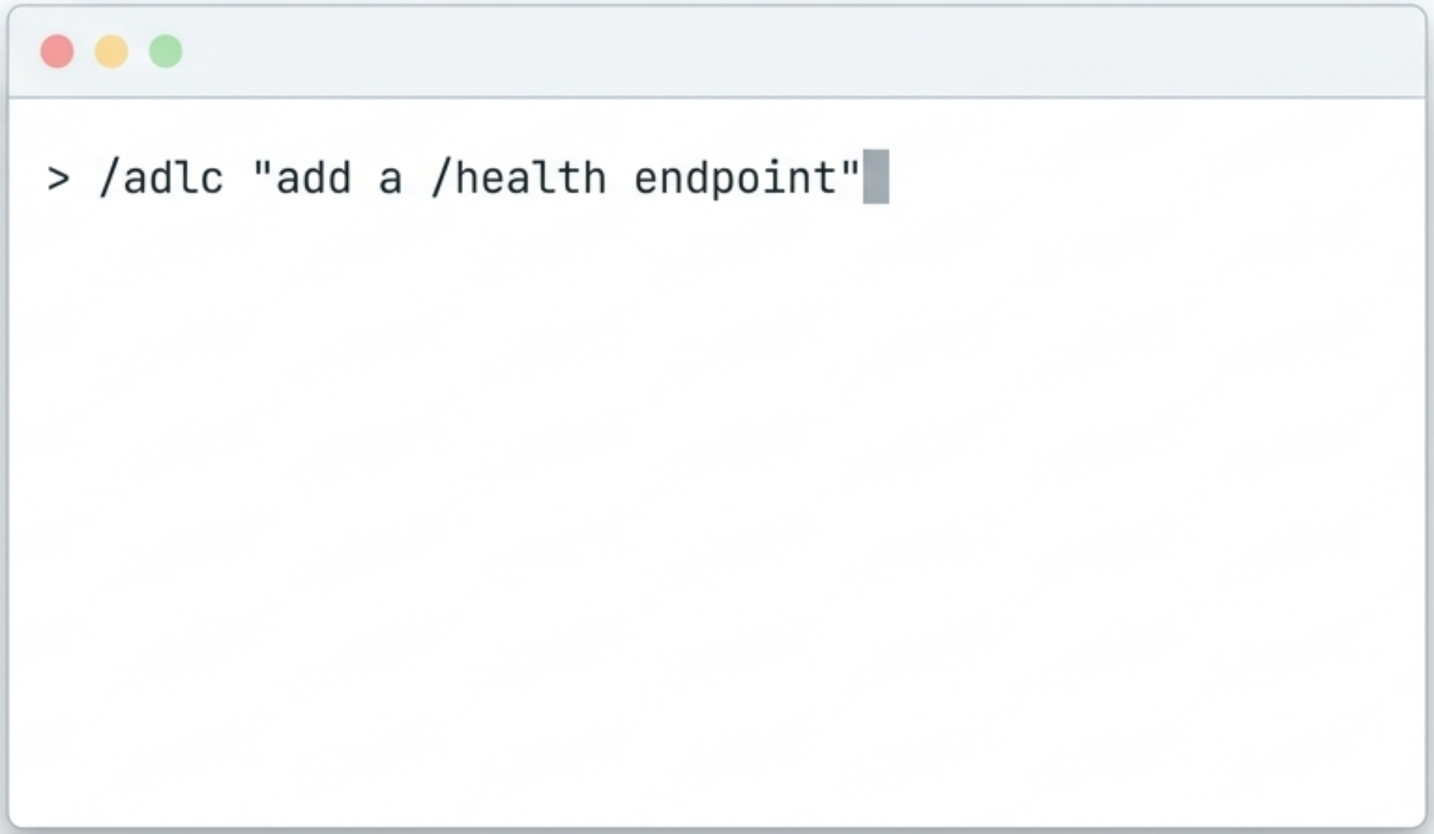


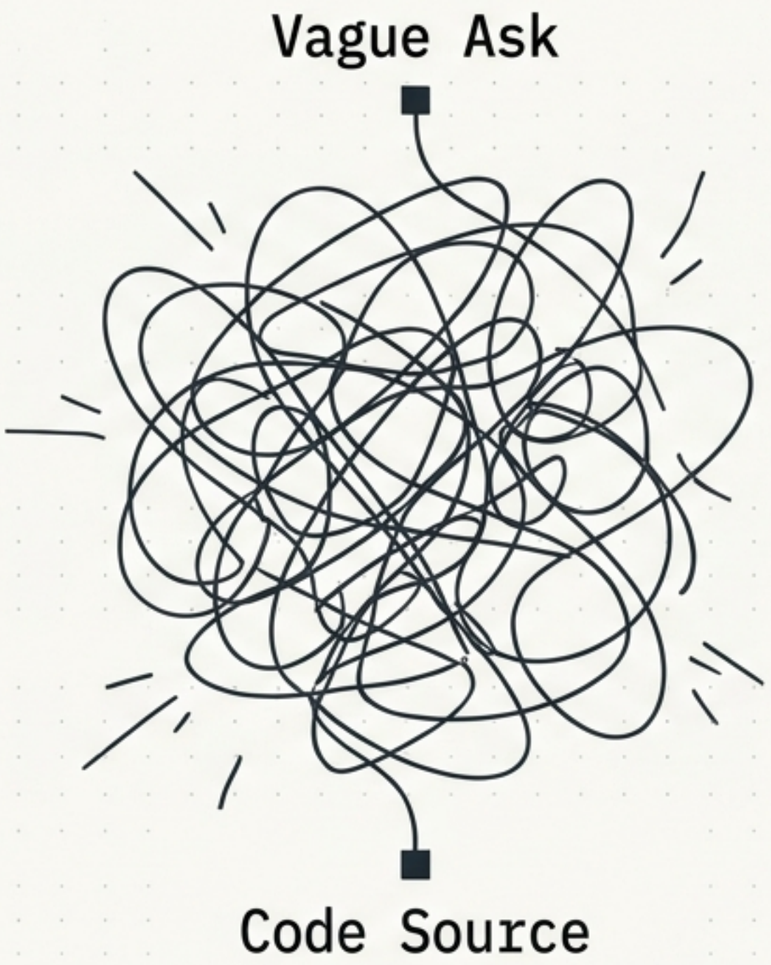
ADLC: Agentic Development Life Cycle

From a one-line request to a reviewed pull request, automatically.

NotebookLM

AI Speed Without SDLC Discipline

Modern AI Coding

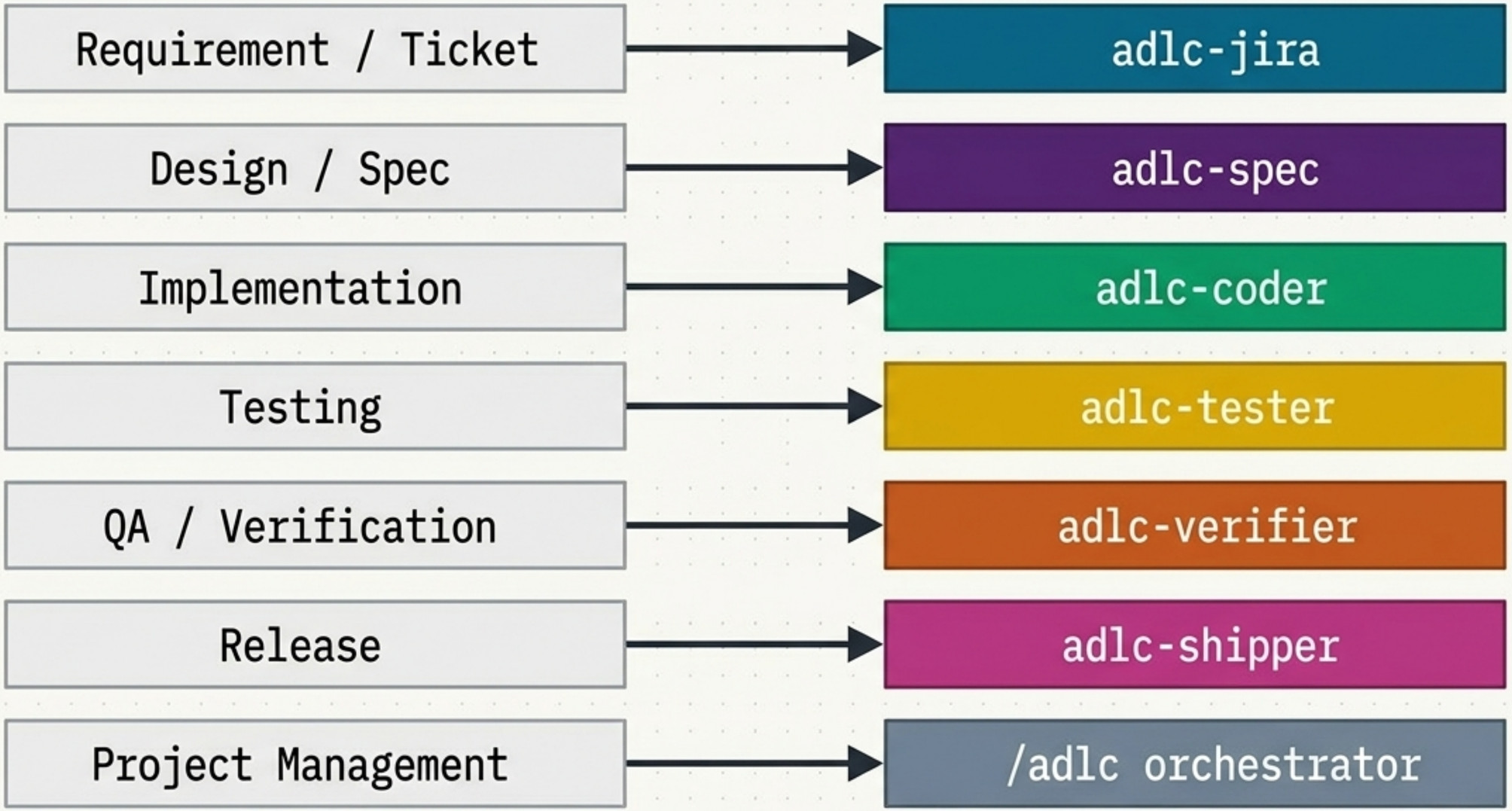


The Missing Discipline

- Unstructured Execution: **Single chat agents** bypass **tickets**, **designs**, and **approval trails**.
- No Planning Gate: **No human checkpoint** to catch a **wrong architectural plan** before **code is written** (making fixes expensive).
- **Vendor Lock-in:** Workflows, prompts, and **credentials** are **hardcoded** to specific tools (Claude Code, Gemini, Cline). Swap the tool, lose the setup.
- **Assertions vs. Proofs: Tests** and **verification** are **ad-hoc**; the AI asserts it works without **deterministic proof**.

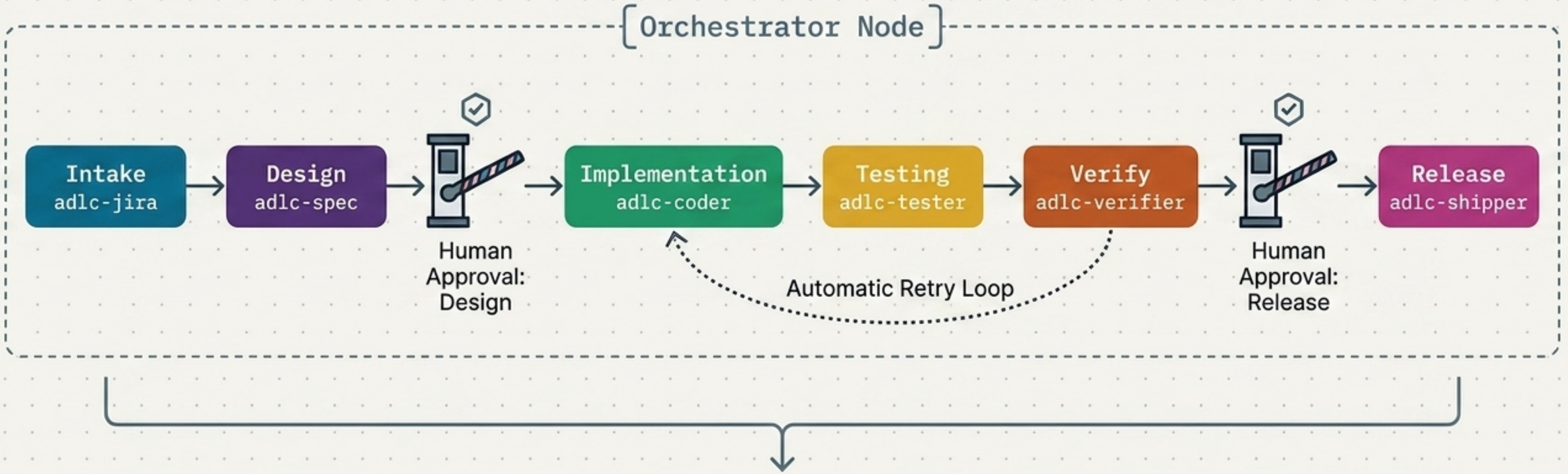
NotebookLM

The Paradigm Shift: Agentic SDLC



NotebookLM

Solution Architecture at a Glance



docs/adlc/<KEY>/
- ticket.md
- spec.md
- state.md

Full Audit Trail:
All generated artifacts are strictly saved under the target project's local directory.

NotebookLM

Meet the Pipeline: Least-Privilege by Design

Agent	Role	Least-Privilege Tools	Produces
adlc-jira	Intake -> ticket	Bash, Read, Write	ticket.md
adlc-spec	Design / spec doc	Read, Grep, Glob, Write	spec.md
adlc-coder	Implement approved spec	Read, Edit, Write, Grep, Glob, Bash	code on a branch
adlc-tester	Tests for acceptance criteria	Read, Edit, Write, Grep, Glob, Bash	test files
adlc-verifier	Run tests/build	Read, Bash, Grep, Glob	verification.md
adlc-shipper	Commit + push	Bash, Read	commit + PR compare URL

Why 6 and not 1?

Containment is safety.

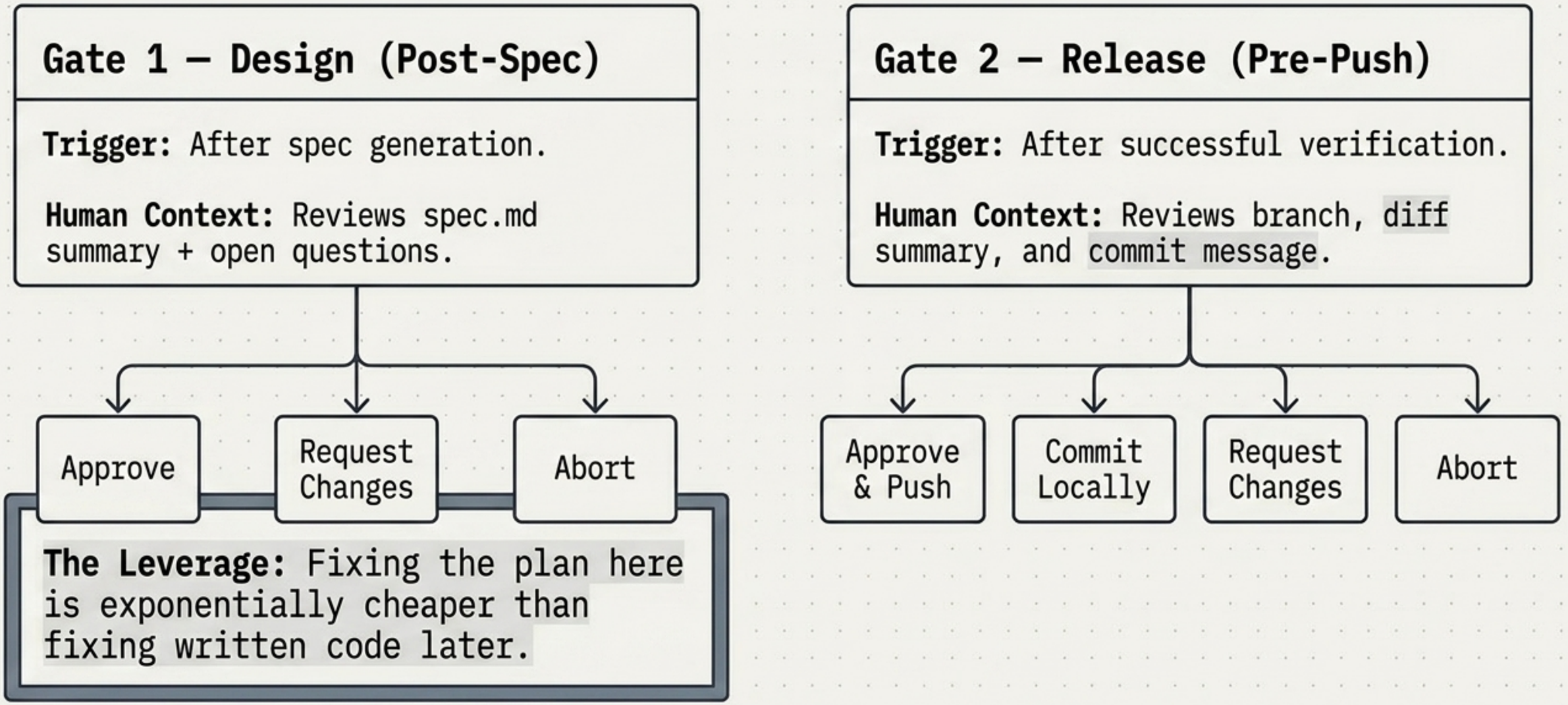
Each agent gets a clean context and restricted powers.

`Spec` and `Verifier` agents literally cannot modify source code.

The `Shipper` can only run `git`.

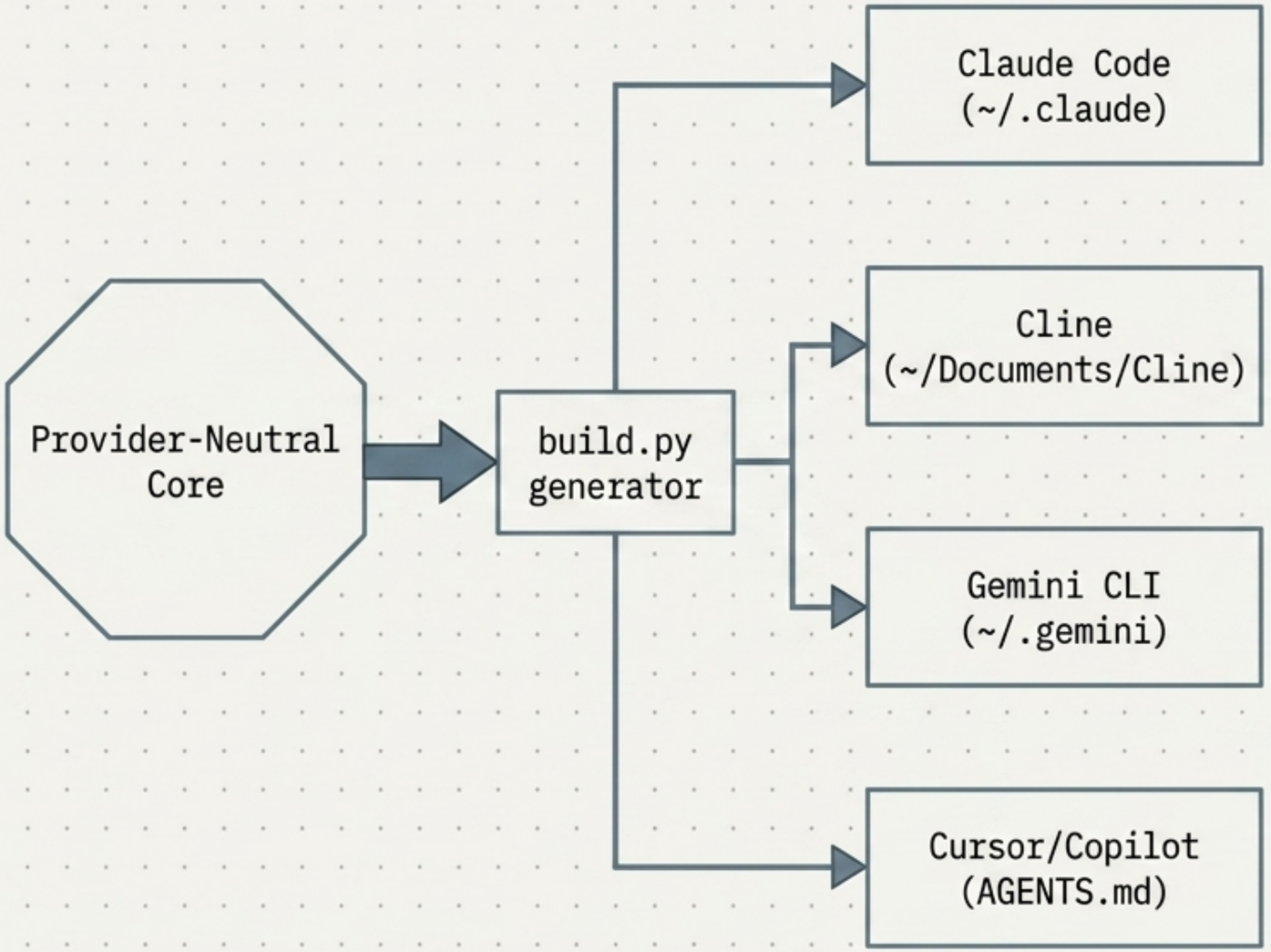
NotebookLM

Human-in-the-Loop: Safety Without Friction



NotebookLM

Universal Portability: Decoupled from the LLM

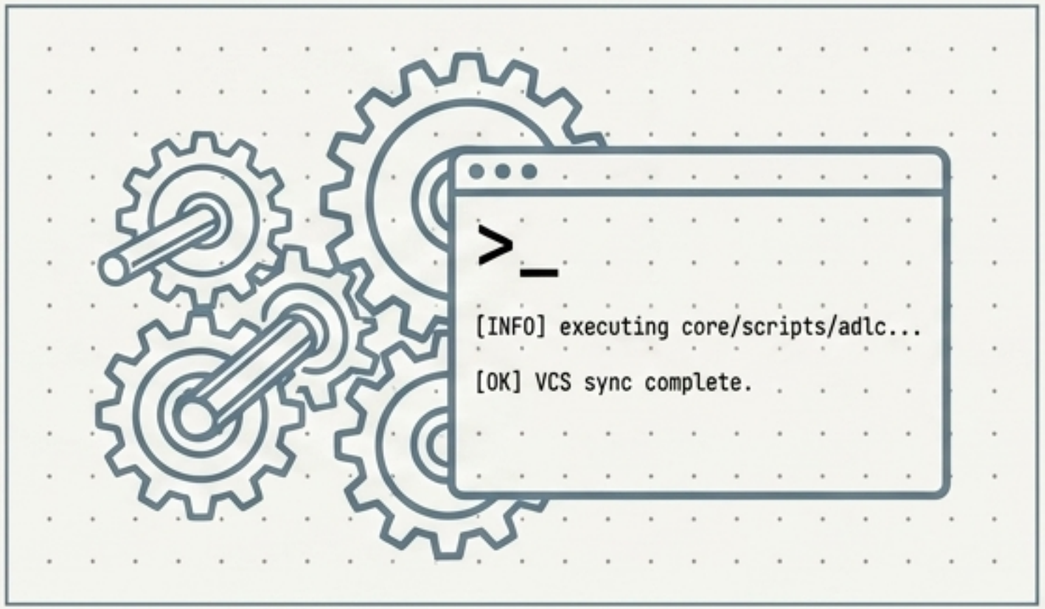


- **Roles, Not Sub-Agents:** The runbook defines 6 self-contained roles. On Claude, they map to real sub-agents. On single-agent hosts, they execute inline gracefully.
- **One Source of Truth:** The core intelligence is neutral. A generator script packages this core into thin adapters for distinct hosts.
- **Anti-Drift Architecture:** Edit core/, run build.py, and every host environment re-syncs automatically. Adapters are never hand-edited.

NotebookLM

The Deterministic Engine (Zero-LLM Operations)

Deterministic Engine

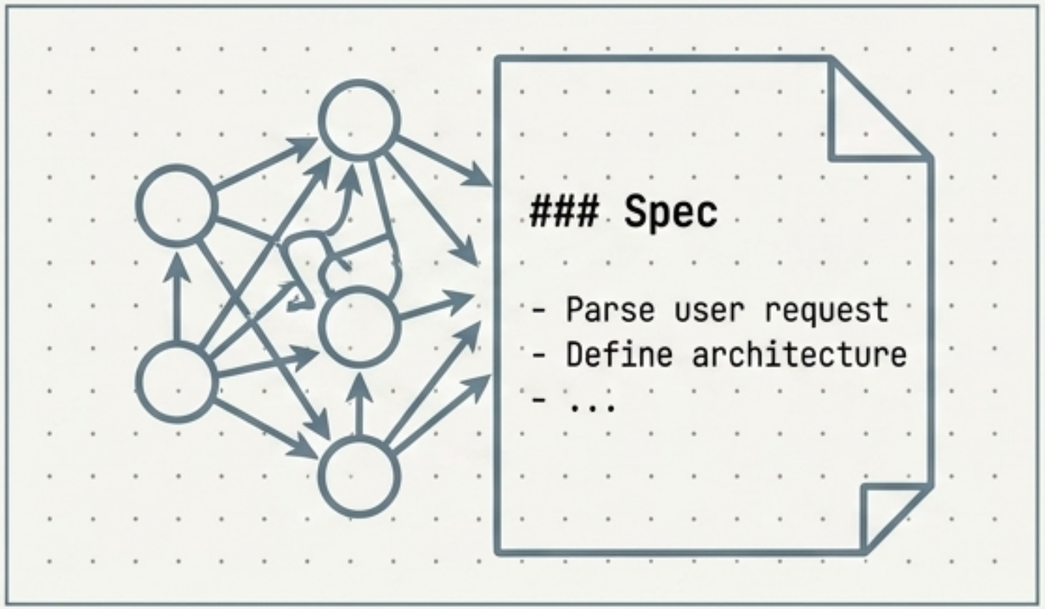


Determinism Lives in Scripts

Mechanical tasks run in core/scripts/adlc and are 100% identical under any model.

Jira ->	Python stdlib (no pip)
VCS ->	Pure Git (no gh dependency)
Cross-OS ->	Native shims (Unix, CMD, PowerShell)

LLM Reasoning

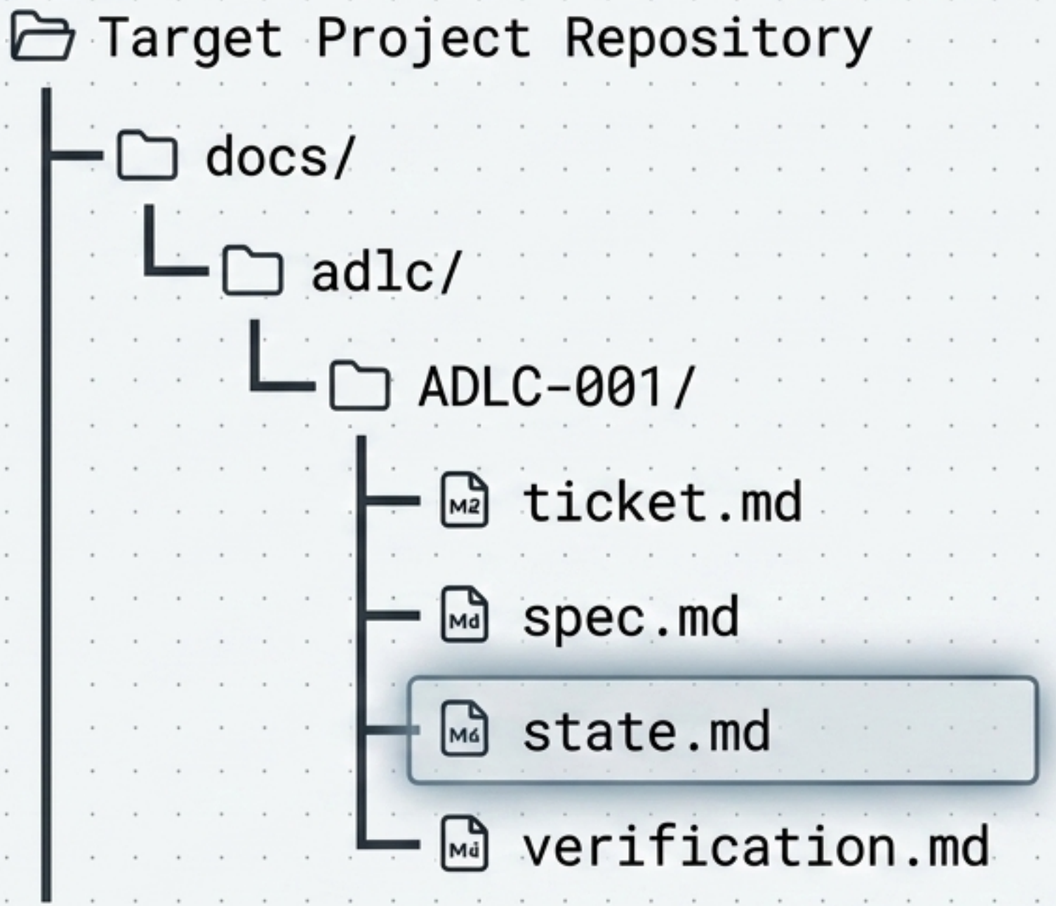


Reasoning Lives in Markdown

The LLM is strictly reserved for non-deterministic intelligence: parsing tickets, writing specs, generating code, and authoring tests.

NotebookLM

State Management & The Audit Trail



Local Permanence

Everything lands **directly** in the **target project repository**, never locked away inside the framework or a cloud web UI.

The Audit Trail

`docs/adlc/<KEY>/` holds the **exact timeline** of tickets, specs, code, and verification reports.

Instant Resumability

`state.md` acts as the system's memory. If the pipeline halts, a developer seamlessly re-runs `/adlc resume <KEY>` to pick up exactly where they left off.

NotebookLM

Six Strict Guarantees of Portability



Shared Install

One framework copy; every local project accesses it. No marketplace required.



No Hardcoded Paths

Artifacts always write relative to the active target project.



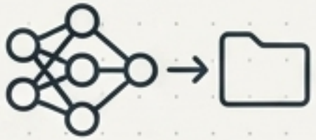
Zero Repo Secrets

Tokens stay in the developer's local shell or a gitignored .env file.



Cross-OS Reliability

.gitattributes forces LF on scripts to prevent CRLF breakage on Linux environments.



Inherited Models

Makes zero assumptions about the host tool's default LLM.



Graceful Degradation

No Jira credentials? Generates local tickets. No git remote? Defaults to local commits.

NotebookLM

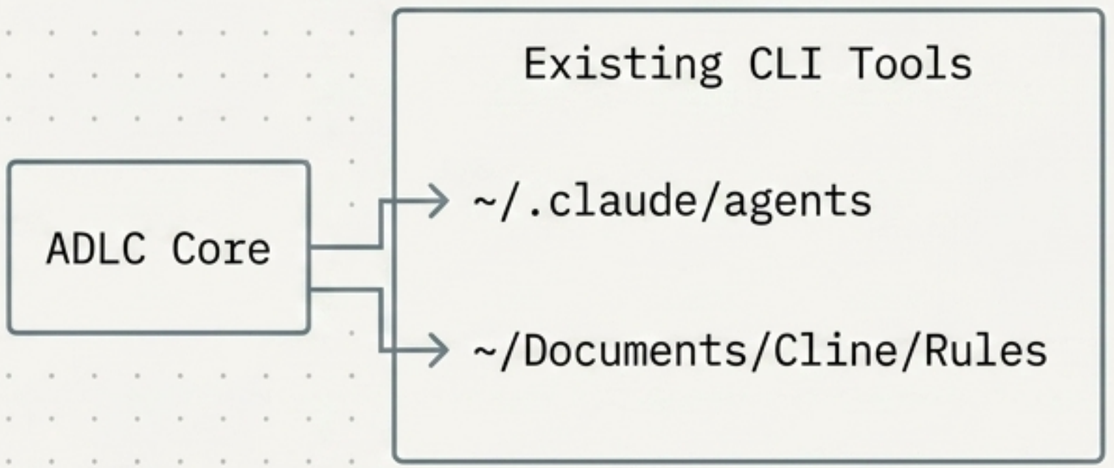
Installation: Zero-Marketplace Philosophy

```
$ ./install.sh

[INFO] Running build.py generator...
[INFO] Setting ADLC_HOME path...

$ cp .env.example .env
```

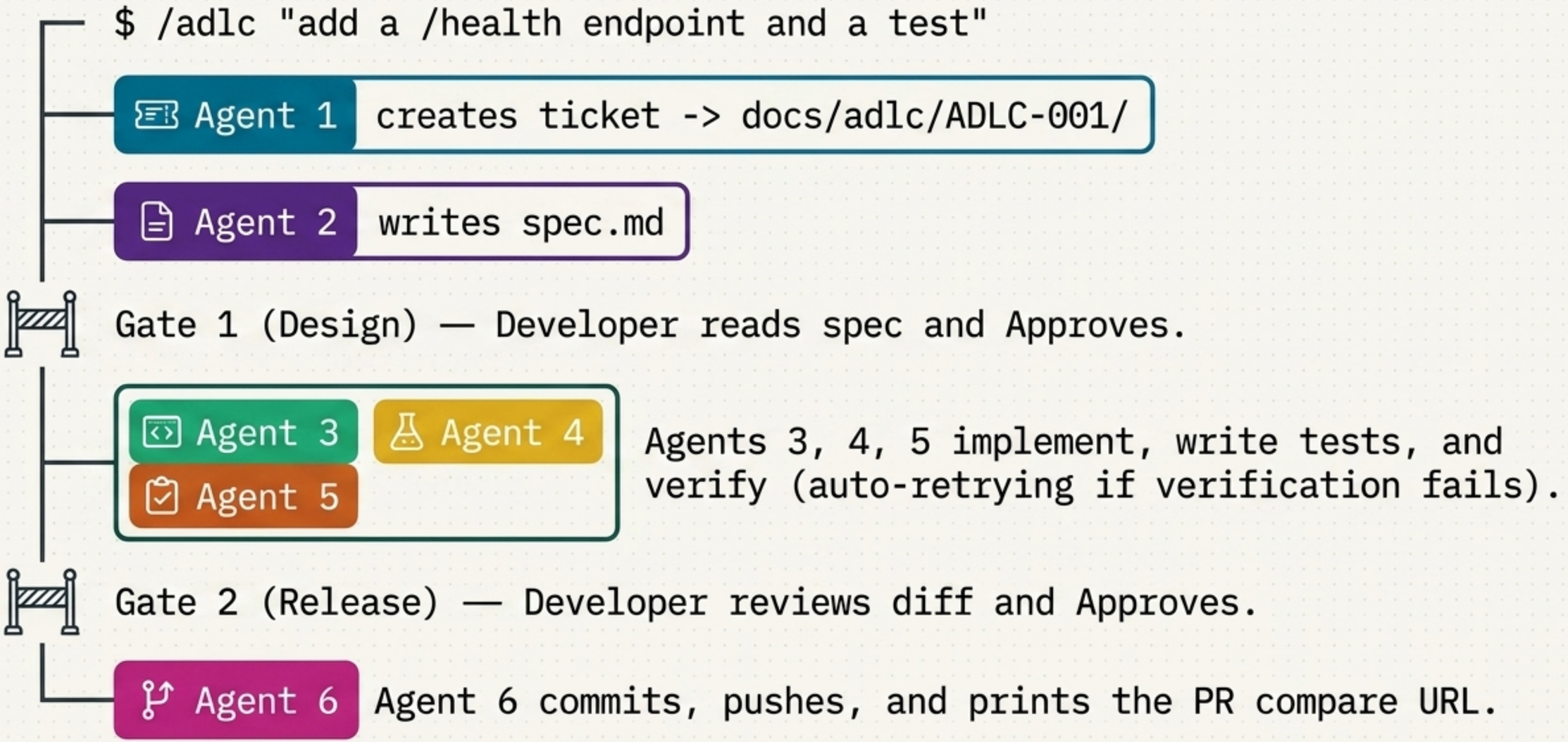
User-Global Injection



Opt-In Integrations: Jira integration requires only a simple .env file copy (JIRA_BASE_URL, JIRA_API_TOKEN). If missing, the pipeline gracefully falls back to local markdown tickets.

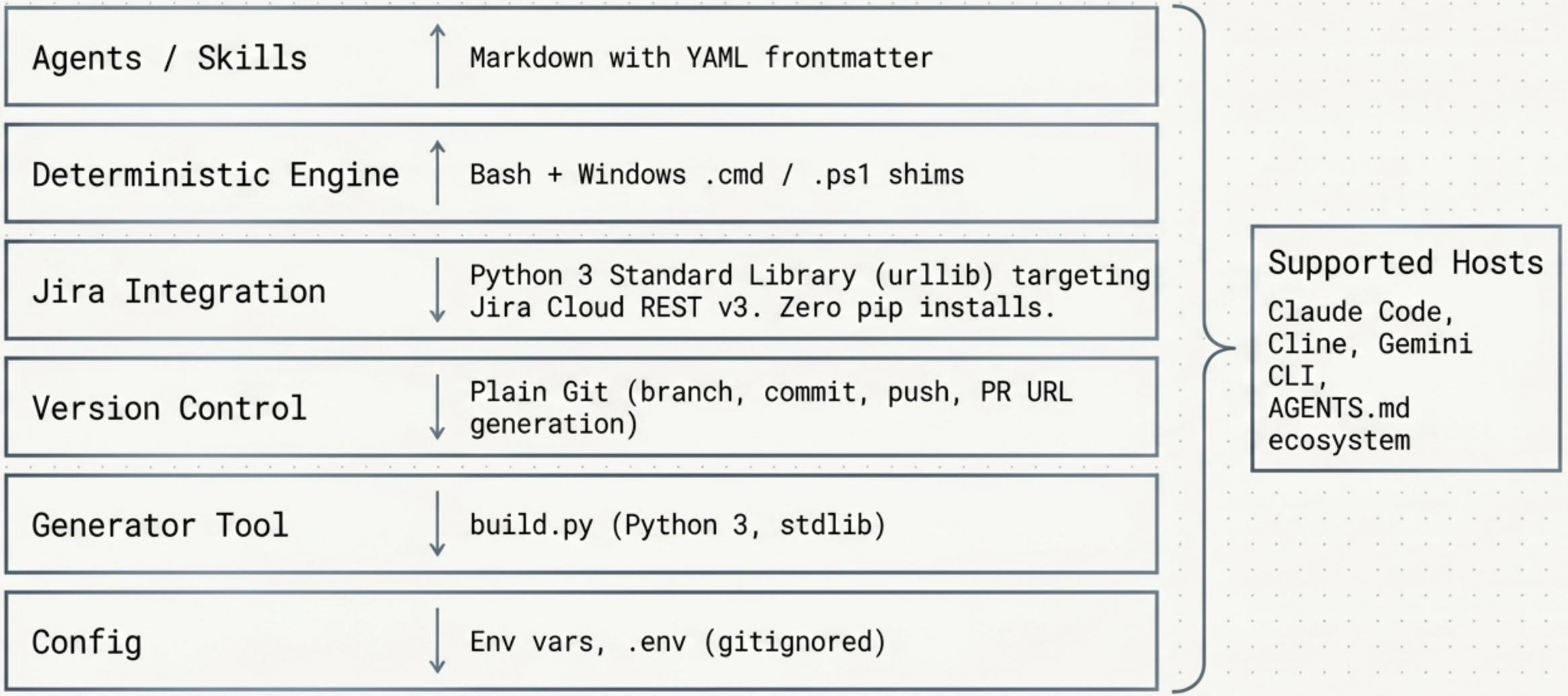
NotebookLM

Usage Walkthrough: From Ask to Pull Request



NotebookLM

Under the Hood: The Minimalist Stack



NotebookLM

The Anti-Drift, Pro-Discipline Paradigm

Core Philosophy

1. Human-in-the-loop at 2 key gates, ensuring safety without constant friction.
2. Containment by construction through least-privilege agent tooling.
3. Reproducible, model-independent workflows driven by deterministic bash/python scripts.
4. Local-first execution requiring no remote access to run natively.

What's Next (Roadmap)



Direct MCP Server wrapping for core/scripts.



Automated GitHub PR creation (replacing compare URLs).



CI Wiring to run the adlc-verifier seamlessly on push.

NotebookLM